TECHNICAL SPECIFICATION

AI Harness

Гибридная спецификация: Skill-Driven + Template-Guarded
Модули, валидация, манифест

Подход: Гибридный (Agent-first + Deterministic validation)

Модули: 8 (L1: 2, L2: 2, L3: 2, Cross: 2)

Версия: 1.0

Июнь 2026

Содержание

1. Принципы гибридного подхода

2. Архитектура: 8 модулей

3. L1: Context System

- 3.1. Модуль: AGENTS.md
- 3.2. Модуль: Instruction Modules

4. L2: Quality & Safety

- 4.1. Модуль: Quality Pipeline
- 4.2. Модуль: Safety Config

5. L3: Memory & Protocol

- 5.1. Модуль: Knowledge Base
- 5.2. Модуль: Session Protocol

6. Cross-cutting: Feedback

- 6.1. Модуль: Skills Registry
- 6.2. Модуль: Feedback Loop

7. Deterministic Validator

8. Манифест (.harness/manifest.json)

9. Три режима работы

10. Пример: полный SKILL.md

11. Матрица: Gen vs Template vs Validate

1. Принципы гибридного подхода

Гибридный ХАРНЕС работает по трем правилам:

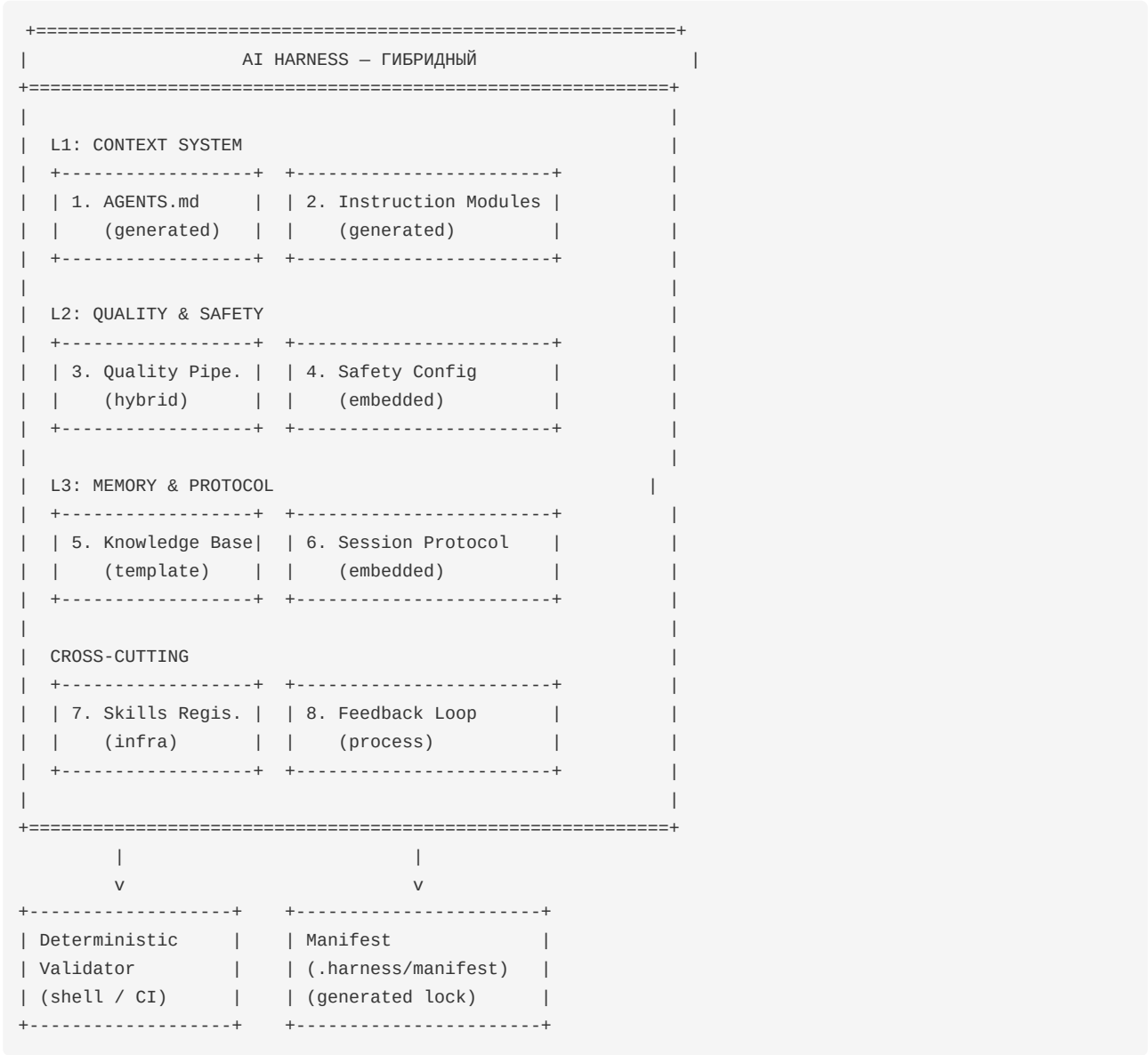
Таблица 1: Правила гибридного подхода

#	Правило	Смысл
1	**Skill описывает, агент генерирует**	SKILL.md содержит декларативные описания модулей. Агент читает skill, анализирует проект, генерирует артефакты.
2	**Security-критичное копируется, не генерируется**	CSP, secrets, auth-конфиги — verified templates. Агент адаптирует переменные, но структура фиксирована.
3	**Все валидируется deterministic checker'ом**	После генерации запускается validator (shell-скрипт). Если проверка не проходит — harness не считается настроенным.

Для каждого модуля определены три аспекта:

- `Generated` — агент генерирует на основе skill и контекста проекта
- `Template` — копируется из verified template с минимальной адаптацией
- `Embedded` — встроено в другой модуль, не существует отдельно
- `VALIDACIYA` — deterministic check, который запускается после генерации

2. Архитектура: 8 модулей



3. L1: Context System

3.1. Модуль: AGENTS.md (Root Manifest)

Тип: Generated **Артефакт:** `./AGENTS.md` **Слой:** L1

Назначение

Корневой файл контекста. Агент читает его при каждом старте сессии. Содержит стек, архитектурные решения, команды, антипаттерны, safety-конфигурацию.

SKILL.md секция (что писать в skill)

```
 ## Модуль: AGENTS.md
НЕОБХОДИМО: AGENTS.md в корне проекта, < 50 строк.
СТРУКТУРА:
  - Stack: [определи из package.json / pyproject.toml / go.mod]
  - Architecture: [monolith | layered | microfrontend]
  - Commands: build, test, lint, typecheck
  - Anti-patterns: список запрещенных практик
  - Safety: запрещенные операции

ВОПРОСЫ ПОЛЬЗОВАТЕЛЮ:
  1. "Какая архитектура проекта?" (если не определяется автоматически)
  2. "Есть ли особые требования к безопасности?"

ГЕНЕРАЦИЯ:
  - Сканируй package.json для dependencies
  - Определи стек: Next.js, React, Vue, Angular, Svelte, Go, Python, etc.
  - Заполни Commands из scripts в package.json
  - Anti-patterns: всегда включай no-any, no-inline-styles, no-console

ВАЛИДАЦИЯ (VALIDACIYA):
  - Файл существует: test -f AGENTS.md
  - Размер < 50 строк: wc -l AGENTS.md
  - Содержит разделы: grep -E "^(## |Stack|Commands|Anti-patterns)" AGENTS.md
```

Пример сгенерированного AGENTS.md

```
 ## Stack
Next.js 15, React 19, TypeScript 5.6, Tailwind CSS, Prisma, PostgreSQL

## Architecture
Layered: UI -> Services -> Data Access. No cross-layer imports.

## Commands
- pnpm dev        # Development server
- pnpm build      # Production build
- pnpm test       # Vitest unit tests
- pnpm lint       # Biome check
- pnpm typecheck  # tsc --noEmit

## Anti-patterns
- No `any` type — always use explicit types
- No inline styles — use Tailwind classes
- No console.log in production code
- No cross-layer imports (UI cannot import from data-access)

## Safety
These operations require explicit user confirmation:
- rm -rf, git push --force, curl | sh
- Editing .env, .env.local files
- Modifying CSP or security headers
- Installing unknown npm packages
```

VALIDACIYA

```
 # --- AGENTS.md validator ---
test -f AGENTS.md || exit 1
[ "$(wc -l < AGENTS.md)" -lt 60 ] || exit 1
grep -q "^## Stack" AGENTS.md || exit 1
grep -q "^## Commands" AGENTS.md || exit 1
grep -q "^## Anti-patterns" AGENTS.md || exit 1
grep -q "^## Safety" AGENTS.md || exit 1
echo "AGENTS.md: PASS"
```

3.2. Модуль: Instruction Modules

Тип: Generated **Артефакт:** `./.claude/instructions/` **Слой:** L1

Назначение

Объединение Path Rules и Skills. Два типа инструкций: `local/` (project-specific) и `shared@version/` (cross-project из registry). Progressive disclosure через globs и phase-триггеры.

SKILL.md секция

```
 ## Модуль: Instruction Modules
НЕОБХОДИМО: директория .claude/instructions/ с поддиректориями local/ и shared/.

LOCAL (project-specific):
  - Создай правила на основе стека проекта
  - react-components.md: правила для React-компонентов
  - api-endpoints.md: правила для API (if backend detected)
  - tests.md: паттерны тестирования

SHARED (cross-project):
  - Подключи через: git subtree add --prefix=.claude/instructions/shared \
      https://github.com/your-org/harness-instructions.git v1.0.0
  - Или создай symlink: ln -s ~/harness/instructions/shared .claude/instructions/shared

ФОРМАТ КАЖДОЙ ИНСТРУКЦИИ:
  ---
  name: [имя]
  type: local | shared
  trigger: glob:**/*.tsx | phase:w2 | manual
  tags: [react, ui, testing]
  ---
  # [Название]
  ## Context
  ## Rules
  ## Examples

ВОПРОСЫ ПОЛЬЗОВАТЕЛЮ:
  1. "Какие домены в проекте?" (UI, API, real-time, data-viz)
  2. "Есть ли shared instruction registry?" (URL или local path)
```

VALIDACIYA

```
 # --- Instruction Modules validator ---
test -d .claude/instructions/local || exit 1
test -d .claude/instructions/shared || exit 1
# Local: хотя бы 2 файла .md
[ "$(find .claude/instructions/local -name "*.md" | wc -l)" -ge 2 ] || exit 1
# Shared: хотя бы 1 файл .md
[ "$(find .claude/instructions/shared -name "*.md" | wc -l)" -ge 1 ] || exit 1
# Формат frontmatter: каждый .md должен начинаться с ---
for f in .claude/instructions/**/*.md; do
  head -1 "$f" | grep -q "^---" || exit 1
done
echo "Instruction Modules: PASS"
```

4. L2: Quality & Safety

4.1. Модуль: Quality Pipeline

Тип: Generated + Template

Артефакт: `lefthook.yml, biome.json, eslint.config.js, .github/workflows/ci.yml` Слой: L2

Назначение

Объединение Lefthook, Custom Linters и CI Gates. Единый pipeline: lint + typecheck + test. Два runtime-режима: local (lefthook) и CI (GitHub Actions). Гибридная генерация: агент выбирает инструменты под стек, а базовая структура — из template.

SKILL.md секция

```
 ## Модуль: Quality Pipeline
НЕОБХОДИМО: единый pipeline lint + typecheck + test для local и CI.

ВЫБОР ИНСТРУМЕНТОВ (агент определяет из стека):
  TypeScript project:
    - Linter: Biome (base) + ESLint (framework-specific)
    - Typecheck: tsc --noEmit
    - Test: vitest (или jest, если уже используется)
  Python project:
    - Linter: ruff
    - Typecheck: mypy
    - Test: pytest
  Go project:
    - Linter: golangci-lint
    - Test: go test

LOCAL (lefthook.yml):
  - Parallel execution: lint | typecheck | test
  - Быстрый feedback: fail fast

CI (.github/workflows/ci.yml):
  - Те же команды, что и в lefthook
  - Parallel jobs: lint, typecheck, test, build
  - Hard gates: PR не мержится без прохождения

TEMPLATE (копируется, агент адаптирует):
  - Базовый lefthook.yml из template/
  - Базовый ci.yml из template/.github/workflows/
  - Агент заменяет VARIABLES на актуальные команды

ВОПРОСЫ ПОЛЬЗОВАТЕЛЮ:
  1. "Какой test runner?" (если не определяется из зависимостей)
  2. "Есть ли существующие workflows?" (merge vs replace)
```

Template: lefthook.yml (базовая структура)

```
# template/lefthook.yml — копируется агентом, адаптирует VARIABLES
pre-commit:
  parallel: true
  commands:
    lint:
      run: {{LINT_CMD}}
    typecheck:
      run: {{TYPECHECK_CMD}}
    test:
      run: {{TEST_CMD}}

pre-push:
  commands:
    build:
      run: {{BUILD_CMD}}
```

Template: ci.yml (базовая структура)

```
# template/.github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: {{INSTALL_CMD}}
      - run: {{LINT_CMD}}
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: {{INSTALL_CMD}}
      - run: {{TEST_CMD}}
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: {{INSTALL_CMD}}
      - run: {{BUILD_CMD}}
```

VALIDACIYA

```
 # --- Quality Pipeline validator ---
# Files exist
test -f lefthook.yml || exit 1
test -f .github/workflows/ci.yml || exit 1
# Linter config exists (at least one)
test -f biome.json || test -f eslint.config.js || test -f pyproject.toml || exit 1
# lefthook has 3+ commands
[ "$(grep -c "run:" lefthook.yml)" -ge 3 ] || exit 1
# CI has 3+ jobs
[ "$(grep -c "runs-on:" .github/workflows/ci.yml)" -ge 3 ] || exit 1
# Syntax check
lefthook validate 2>/dev/null || true
echo "Quality Pipeline: PASS"
```

4.2. Модуль: Safety Config

Тип: Embedded **Артефакт:** Раздел в `AGENTS.md` **Слой:** L2

Назначение

Бывший PreToolUse Hooks. Блокировка опасных операций встраивается как раздел в AGENTS.md, а не как отдельный файл. Агент читает этот раздел при старте и следует ему.

SKILL.md секция

```
 ## Модуль: Safety Config
ТИП: Embedded (раздел в AGENTS.md)
НЕОБХОДИМО: раздел ## Safety в AGENTS.md.

СОДЕРЖАНИЕ (всегда фиксировано, не генерируется):
  These operations require explicit user confirmation:
  - rm -rf, git push --force, curl | sh
  - Editing .env, .env.local, .env.production files
  - Modifying CSP headers, CORS config, auth middleware
  - Installing packages with postinstall scripts (verify first)
  - Exposing ports, changing firewall rules
  - Modifying database schemas with data loss risk

ВАЛИДАЦИЯ: AGENTS.md содержит раздел "## Safety" с 5+ запретами.
```

VALIDACIYA

```
 # --- Safety Config validator ---
grep -q "^## Safety" AGENTS.md || exit 1
# Минимум 5 запретов (строк с "-")
[ "$(sed -n '/^## Safety/,/^## /p' AGENTS.md | grep -c "^  -")" -ge 5 ] || exit 1
echo "Safety Config: PASS"
```

5. L3: Memory & Protocol

5.1. Модуль: Knowledge Base

Тип: Template **Артефакт:** `docs/knowledge/` **Слой:** L3

Назначение

Объединение ADR Archive и Lessons Log. Единая директория с тегируемыми записями трех типов: `decision`, `lesson`, `pattern`. Шаблоны записей — из template, содержимое — пишут агент и разработчики.

SKILL.md секция

```
 ## Модуль: Knowledge Base
НЕОБХОДИМО: директория docs/knowledge/ с шаблонами.

ШАБЛОНЫ (копируются из template/, агент не генерирует):
  - template/knowledge/decision.md — для архитектурных решений
  - template/knowledge/lesson.md — для уроков из ошибок
  - template/knowledge/pattern.md — для повторяющихся паттернов

ПРАВИЛА ИМЕНОВАНИЯ:
  - Формат: NNN-descriptive-slug.md
  - Примеры: 001-ssr-over-csr.md, 002-no-blur-hash-ssr.md

АГЕНТ СОЗДАЕТ при ошибках:
  - Когда lint не проходит -> lesson
  - Когда принимает архитектурное решение -> decision
  - Когда находит повторяющийся паттерн -> pattern
```

Templates (копируются, не генерируются)

```
 ---
# template/knowledge/decision.md
type: decision
tags: [architecture, performance, security]
date: YYYY-MM-DD
status: accepted | deprecated | superseded by NNN
---
# Title

## Context
What is the problem or opportunity?

## Decision
What was decided and why?

## Consequences
Positive and negative impacts.

## Alternatives Considered
Why were they rejected?

---
# template/knowledge/lesson.md
type: lesson
tags: [react, ssr, testing]
date: YYYY-MM-DD
severity: high | medium | low
---
# Title

## Problem
What went wrong?

## Root Cause
Why did it happen?

## Solution
How was it fixed?

## Prevention
How to prevent in the future?

---
# template/knowledge/pattern.md
type: pattern
tags: [state-management, component-design]
date: YYYY-MM-DD
---
# Title

## Context
When to apply this pattern?

## Implementation
How to implement it?
```

VALIDACIYA

```
 # --- Knowledge Base validator ---
test -d docs/knowledge || exit 1
# Шаблоны на месте
test -f docs/knowledge/.template-decision.md || exit 1
test -f docs/knowledge/.template-lesson.md || exit 1
test -f docs/knowledge/.template-pattern.md || exit 1
# Каждый .md (кроме шаблонов) имеет frontmatter с type:
for f in docs/knowledge/*.md; do
  [[ "$f" == *".template-"* ]] && continue
  head -1 "$f" | grep -q "^---" || exit 1
done
echo "Knowledge Base: PASS"
```

5.2. Модуль: Session Protocol

Тип: Embedded **Артефакт:** Раздел в `AGENTS.md` + `feature_list.json` **Слой:** L3

Назначение

Объединение Git Checkpoints и Context Reset Protocol. Описывает, как агент управляет сессией: когда коммитить, когда сбрасывать контекст, как передавать состояние между сессиями.

SKILL.md секция

```
 ## Модуль: Session Protocol
ТИП: Embedded (раздел в AGENTS.md) + feature_list.json
НЕОБХОДИМО: раздел ## Session Protocol в AGENTS.md + feature_list.json.

ПРОТОКОЛ (встраивается в AGENTS.md):
  ## Session Protocol
  1. Before starting: read feature_list.json to understand state
  2. Per feature: create conventional commit ("feat: F02 image pipeline")
  3. On context switch: /clear + pass structured context (current feature, decisions)
  4. On completion: update feature_list.json + record lesson if needed

feature_list.json (агент генерирует структуру, обновляет по ходу):
  {
    "project": "...",
    "features": [
      {"id": "F01", "title": "Auth", "status": "completed", "commit": "abc123"}
    ],
    "tech_debt": [],
    "active_decisions": [],
    "last_updated": "2026-06-02"
  }
```

VALIDACIYA

```
 # --- Session Protocol validator ---
grep -q "^## Session Protocol" AGENTS.md || exit 1
test -f feature_list.json || exit 1
# feature_list.json — валидный JSON
python3 -c "import json; json.load(open('feature_list.json'))" || exit 1
echo "Session Protocol: PASS"
```

6. Cross-cutting: Feedback

6.1. Модуль: Skills Registry

Тип: Template **Артефакт:** `~/harness/instructions/shared/` или git repo **Слой:** Cross

Назначение

Центральный репозиторий shared-инструкций. Не генерируется агентом — это инфраструктурный репозиторий, который администрируется отдельно.

SKILL.md секция

```
 ## Модуль: Skills Registry
ТИП: Infrastructure (не генерируется агентом)
НЕОБХОДИМО: доступ к shared instructions.

ПОДКЛЮЧЕНИЕ:
  git subtree add --prefix=.claude/instructions/shared \
    https://github.com/your-org/harness-instructions.git v1.0.0

ИЛИ symlink:
  ln -s ~/harness/instructions/shared .claude/instructions/shared

ВЕРСИОНИРОВАНИЕ: semver tags (v1.0.0, v1.1.0)
ОБНОВЛЕНИЕ: git subtree pull для обновления
```

VALIDACIYA

```
 # --- Skills Registry validator ---
test -d .claude/instructions/shared || exit 1
# Shared содержит хотя бы 1 SKILL.md
[ "$(find .claude/instructions/shared -name "SKILL.md" | wc -l)" -ge 1 ] || exit 1
echo "Skills Registry: PASS"
```

6.2. Модуль: Feedback Loop

Тип: Generated **Артефакт:** Процесс, описанный в `AGENTS.md` **Слой:** Cross

Назначение

Объединение Improvement Loop и Evaluator. Цикл обратной связи: агент ошибается → pipeline ловит → записывается lesson → instruction module обновляется. Описывается как процесс в AGENTS.md.

SKILL.md секция

```
 ## Модуль: Feedback Loop
ТИП: Process (описание в AGENTS.md)
НЕОБХОДИМО: раздел ## Feedback Loop в AGENTS.md.

ЦИКЛ:
  1. Agent makes mistake (uses `any`, forgets test, etc.)
  2. Quality Pipeline catches it (lint fails, test fails)
  3. Agent records Lesson in docs/knowledge/NNN-slug.md
  4. Agent updates relevant Instruction Module (new rule)
  5. On next session: Agent reads updated instructions

EVALUATION (еженедельно):
  - Count: lessons recorded, instructions updated, lint violations
  - Trend: lessons should decrease over time as harness improves
```

VALIDACIYA

```
 # --- Feedback Loop validator ---
grep -q "^## Feedback Loop" AGENTS.md || exit 1
# Хотя бы 1 lesson или decision в Knowledge Base
[ "$(find docs/knowledge -name "*.md" ! -name ".template-*" | wc -l)" -ge 1 ] || echo "WARN: no
knowledge entries yet"
echo "Feedback Loop: PASS"
```

7. Deterministic Validator

Validator — единственный кастомный компонент в гибридном ХАРНЕСЕ. Он запускается после генерации и проверяет, что все модули настроены корректно. Работает без LLM.

7.1. Архитектура валидатора

```
 +----------------------------------------+
|  harness-validate (shell script)        |
|                                         |
|  Phase 1: Structure checks              |
|    - All required files exist           |
|    - All required directories exist     |
|    - File sizes within bounds           |
|                                         |
|  Phase 2: Content checks                |
|    - Required sections present          |
|    - Required keywords present          |
|    - JSON/YAML syntax valid             |
|                                         |
|  Phase 3: Integration checks            |
|    - Lint passes (biome/eslint)         |
|    - Typecheck passes (tsc/mypy)        |
|    - Lefthook config valid              |
|                                         |
|  Output: manifest.json + report         |
+----------------------------------------+
```

7.2. Полный скрипт валидатора

```
 #!/bin/bash
# harness-validate.sh — deterministic harness validator
# Runs WITHOUT LLM. Pure shell.

set -e
ERRORS=0
WARNINGS=0
MANIFEST=".harness/manifest.json"

log_pass() { echo "  [PASS] $1"; }
log_fail() { echo "  [FAIL] $1"; ERRORS=$((ERRORS+1)); }
log_warn() { echo "  [WARN] $1"; WARNINGS=$((WARNINGS+1)); }

# ============================================
# PHASE 1: Structure Checks
# ============================================
echo "=== Phase 1: Structure ==="

# L1: AGENTS.md
if [ -f "AGENTS.md" ]; then
  LINES=$(wc -l < AGENTS.md)
  if [ "$LINES" -lt 60 ]; then
    log_pass "AGENTS.md exists ($LINES lines)"
  else
    log_fail "AGENTS.md too long ($LINES lines, max 60)"
  fi
else
  log_fail "AGENTS.md missing"
fi

# L1: Instruction Modules
if [ -d ".claude/instructions/local" ] && [ -d ".claude/instructions/shared" ]; then
  LOCAL_COUNT=$(find .claude/instructions/local -name "*.md" | wc -l)
  SHARED_COUNT=$(find .claude/instructions/shared -name "*.md" | wc -l)
  if [ "$LOCAL_COUNT" -ge 2 ] && [ "$SHARED_COUNT" -ge 1 ]; then
    log_pass "Instruction Modules: $LOCAL_COUNT local, $SHARED_COUNT shared"
  else
    log_fail "Instruction Modules: need 2+ local, 1+ shared"
  fi
else
  log_fail "Instruction Modules directories missing"
fi

# L2: Quality Pipeline
if [ -f "lefthook.yml" ]; then
  CMD_COUNT=$(grep -c "run:" lefthook.yml || true)
  if [ "$CMD_COUNT" -ge 3 ]; then
    log_pass "lefthook.yml with $CMD_COUNT commands"
  else
    log_fail "lefthook.yml: need 3+ commands, got $CMD_COUNT"
  fi
else
  log_fail "lefthook.yml missing"
fi

if [ -f ".github/workflows/ci.yml" ]; then
  JOB_COUNT=$(grep -c "runs-on:" .github/workflows/ci.yml || true)
```

```
 log_pass "CI workflow with $JOB_COUNT jobs"
  else
    log_fail "CI workflow: need 3+ jobs, got $JOB_COUNT"
  fi
else
  log_fail "CI workflow missing"
fi

# Linter config
if [ -f "biome.json" ] || [ -f "eslint.config.js" ] || [ -f "pyproject.toml" ]; then
  log_pass "Linter config present"
else
  log_fail "No linter config found"
fi

# L3: Knowledge Base
if [ -d "docs/knowledge" ]; then
  if [ -f "docs/knowledge/.template-decision.md" ] && \
     [ -f "docs/knowledge/.template-lesson.md" ]; then
    log_pass "Knowledge Base with templates"
  else
    log_warn "Knowledge Base: templates missing"
  fi
else
  log_fail "docs/knowledge directory missing"
fi

# L3: feature_list.json
if [ -f "feature_list.json" ]; then
  if python3 -c "import json; json.load(open('feature_list.json'))" 2>/dev/null; then
    log_pass "feature_list.json is valid JSON"
  else
    log_fail "feature_list.json: invalid JSON"
  fi
else
  log_fail "feature_list.json missing"
fi

# ============================================
# PHASE 2: Content Checks
# ============================================
echo ""
echo "=== Phase 2: Content ==="

grep -q "^## Stack" AGENTS.md && log_pass "AGENTS.md: Stack section" || \
  log_fail "AGENTS.md: Stack section missing"

grep -q "^## Commands" AGENTS.md && log_pass "AGENTS.md: Commands section" || \
  log_fail "AGENTS.md: Commands section missing"

grep -q "^## Anti-patterns" AGENTS.md && log_pass "AGENTS.md: Anti-patterns section" || \
  log_fail "AGENTS.md: Anti-patterns section missing"

grep -q "^## Safety" AGENTS.md && log_pass "AGENTS.md: Safety section" || \
  log_fail "AGENTS.md: Safety section missing"

grep -q "^## Session Protocol" AGENTS.md && log_pass "AGENTS.md: Session Protocol section" || \
```

```
 grep -q "^## Feedback Loop" AGENTS.md && log_pass "AGENTS.md: Feedback Loop section" || \
  log_warn "AGENTS.md: Feedback Loop section missing"

# Safety: minimum 5 rules
SAFETY_RULES=$(sed -n '/^## Safety/,/^## /p' AGENTS.md | grep -c "^  -" || true)
if [ "$SAFETY_RULES" -ge 5 ]; then
  log_pass "Safety: $SAFETY_RULES rules"
else
  log_fail "Safety: need 5+ rules, got $SAFETY_RULES"
fi

# ============================================
# PHASE 3: Integration Checks
# ============================================
echo ""
echo "=== Phase 3: Integration ==="

# Lefthook validation (if installed)
if command -v lefthook &>/dev/null; then
  if lefthook validate 2>/dev/null; then
    log_pass "lefthook config valid"
  else
    log_warn "lefthook validate failed (may be OK)"
  fi
else
  log_warn "lefthook not installed, skipping validation"
fi

# ============================================
# GENERATE MANIFEST
# ============================================
echo ""
echo "=== Manifest ==="

cat > "$MANIFEST" << EOF
{
  "validated_at": "$(date -u +%Y-%m-%dT%H:%M:%SZ)",
  "version": "1.0",
  "errors": $ERRORS,
  "warnings": $WARNINGS,
  "modules": {
    "agents_md": { "file": "AGENTS.md", "lines": ${LINES:-0} },
    "instruction_modules": { "local": ${LOCAL_COUNT:-0}, "shared": ${SHARED_COUNT:-0} },
     "quality_pipeline": { "lefthook": $(test -f lefthook.yml && echo "true" || echo "false"), "ci":
$(test -f .github/workflows/ci.yml && echo "true" || echo "false") },
      "knowledge_base": { "dir": "docs/knowledge", "templates": $(test -f docs/knowledge/.template-
decision.md && echo "true" || echo "false") },
    "session_protocol": { "feature_list": $(test -f feature_list.json && echo "true" || echo "false") }
  }
}
EOF

echo "Manifest written to $MANIFEST"

# ============================================
# SUMMARY
# ============================================
```

```
 echo "  ERRORS:   $ERRORS"
echo "  WARNINGS: $WARNINGS"
echo "======================================="

exit $ERRORS
```

> **Ключевой принцип**
>
> Validator — единственный кастомный компонент. Он не зависит от LLM, не зависит от модели, работает в CI. Если validator проходит — harness считается валидным, независимо от того, кто и как его генерировал.

8. Манифест (.harness/manifest.json)

Манифест — lock-файл сгенерированного harness'а. Создается validator'ом после каждой генерации или обновления. Используется для аудита и cross-project consistency.

8.1. Структура манифеста

```
{
  "validated_at": "2026-06-02T14:30:00Z",
  "version": "1.0",
  "skill_version": "1.2.0",
  "errors": 0,
  "warnings": 0,
  "generated_by": "claude-sonnet-4",
  "project": {
    "stack": "nextjs-react-ts",
    "detected_from": "package.json"
  },
  "modules": {
    "agents_md": {
      "file": "AGENTS.md",
      "lines": 42,
      "sha256": "a3f7c2...",
       "sections": ["Stack", "Architecture", "Commands", "Anti-patterns", "Safety", "Session Protocol",
"Feedback Loop"]
    },
    "instruction_modules": {
      "local_dir": ".claude/instructions/local",
      "local_files": ["react-components.md", "api-endpoints.md", "tests.md"],
      "shared_dir": ".claude/instructions/shared",
      "shared_version": "v1.2.0",
      "shared_files": ["react-component/SKILL.md", "api-endpoint/SKILL.md"]
    },
    "quality_pipeline": {
      "lefthook": { "file": "lefthook.yml", "commands": ["lint", "typecheck", "test"] },
      "ci": { "file": ".github/workflows/ci.yml", "jobs": ["lint", "test", "build"] },
      "linter": { "tool": "biome", "config": "biome.json" }
    },
    "knowledge_base": {
      "dir": "docs/knowledge",
      "templates": [".template-decision.md", ".template-lesson.md", ".template-pattern.md"]
    },
    "session_protocol": {
      "feature_list": "feature_list.json",
      "features_count": 5
    }
  }
}
```

8.2. Назначение манифеста

Таблица 2: Назначение манифеста

Сценарий	Как используется
Аудит	"Что именно создал агент?" — ответ в manifest.json
Сравнение проектов	Два manifest'а — diff для понимания разницы в harness'е
CI gate	PR не проходит, если manifest не обновлен
Rollback	git tag + manifest = точка отката
Cross-project sync	Renovate обновляет skill → новый setup → новый manifest

9. Три режима работы

9.1. Interactive Setup (первичная настройка)

```
 +-----------------------------------------+
|  1. Агент читает SKILL.md                |
|  2. Агент сканирует проект (package.json) |
|  3. Агент задает уточняющие вопросы      |
|  4. Пользователь отвечает                |
|  5. Агент генерирует артефакты           |
|  6. Агент копирует templates             |
|  7. Агент запускает validator            |
|  8. Validator создает manifest           |
|  9. Агент выводит отчет                  |
+-----------------------------------------+

Пример диалога:
  Агент: "Определил стек: Next.js 15 + React 19 + TypeScript."
  Агент: "Есть ли SSR? [auto-detected: yes]"
  Агент: "Какой test runner? (vitest/jest/playwright) [auto-detected: vitest]"
  Агент: "Есть ли shared instruction registry? [URL или skip]"
  Агент: "Генерирую AGENTS.md, Quality Pipeline, Knowledge Base..."
  Агент: "Validator: 0 errors, 1 warning. Harness настроен."
  Агент: "Manifest создан: .harness/manifest.json"
```

9.2. Sync (обновление при новой версии skill)

```
 +-----------------------------------------+
|  1. Новая версия SKILL.md (git pull)     |
|  2. Агент читает diff (что изменилось)   |
|  3. Агент показывает изменения           |
|  4. Пользователь подтверждает            |
|  5. Агент применяет изменения            |
|  6. Validator проверяет                  |
|  7. Manifest обновляется                 |
+-----------------------------------------+

Пример:
  Агент: "Skill обновлен: v1.1.0 -> v1.2.0"
  Агент: "Изменения: +правило 'no-magic-numbers', +шаблон lesson.md"
  Агент: "Применить? [y/n]"
  ...применяет...
  Агент: "Validator: 0 errors. Harness обновлен."
```

9.3. CI-only (deterministic check без LLM)

```
 +-----------------------------------------+
|  CI trigger: push / pull_request          |
|  -> Run harness-validate.sh               |
|  -> No LLM involved                       |
|  -> Pure shell, deterministic             |
|  -> Report as CI annotations              |
+-----------------------------------------+

.github/workflows/harness-validate.yml:
  name: Harness Validation
  on: [push, pull_request]
  jobs:
    validate:
      runs-on: ubuntu-latest
      steps:
        - uses: actions/checkout@v4
        - run: ./.harness/harness-validate.sh
        # FAIL if validator returns non-zero
```

10. Пример: полный SKILL.md

Ниже — пример рабочего SKILL.md для гибридного ХАРНЕСА. Один файл заменяет 17 модулей оригинальной архитектуры.

```
 # AI Harness Skill v1.0
# Один skill = полный harness. Агент читает, генерирует, валидирует.

## Принципы
1. Я описываю модули декларативно (что нужно).
2. Ты генерируешь артефакты (как именно — зависит от стека).
3. Security-критичное копируется из templates/.
4. После генерации запускаешь ./.harness/harness-validate.sh.
5. Если validator падает — чини и перезапускай.

## Setup Questions (задай пользователю)
1. "Какой стек проекта? [auto-detect from package.json]"
2. "Есть ли SSR? [auto-detect]"
3. "Monorepo или single repo? [auto-detect]"
4. "Есть ли shared instruction registry? [URL или skip]"
5. "Особые требования безопасности? [default / HIPAA / PCI / custom]"

---

## Модуль: AGENTS.md
НЕОБХОДИМО: файл AGENTS.md в корне, < 50 строк.
ШАБЛОН: template/AGENTS.md (копируй, адаптируй переменные)

СТРУКТУРА:
  ## Stack
  [определи из package.json: Next.js X, React Y, TS Z, ...]

  ## Architecture
  [layered | monolith | mfe | auto-detect from structure]

  ## Commands
  [из scripts в package.json: dev, build, test, lint, typecheck]

  ## Anti-patterns
  - No `any` type
  - No inline styles
  - No console.log in production
  - [добавь специфичные для стека]

  ## Safety
  These operations require explicit user confirmation:
  - rm -rf, git push --force, curl | sh
  - Editing .env files
  - Modifying CSP or security headers
  - Installing packages with postinstall scripts
  - Modifying database schemas with data loss risk

  ## Session Protocol
  1. Before starting: read feature_list.json
  2. Per feature: conventional commit ("feat: F02 title")
  3. On context switch: /clear + pass structured context
  4. On completion: update feature_list.json

  ## Feedback Loop
  When mistake caught by pipeline:
  1. Record Lesson in docs/knowledge/NNN-slug.md
  2. Update relevant instruction module
```

```
 VALIDACIYA:
   test -f AGENTS.md
   [ "$(wc -l < AGENTS.md)" -lt 60 ]
   grep -q "^## Stack" AGENTS.md
   grep -q "^## Commands" AGENTS.md
   grep -q "^## Anti-patterns" AGENTS.md
   grep -q "^## Safety" AGENTS.md
   grep -q "^## Session Protocol" AGENTS.md

---

## Модуль: Instruction Modules
НЕОБХОДИМО: .claude/instructions/local/ и .claude/instructions/shared/

LOCAL (генерируешь на основе стека):
  - react-components.md (если React/Next.js)
  - api-endpoints.md (если backend routes detected)
  - tests.md (всегда)
  - [другие на основе доменов проекта]

SHARED (подключаешь):
  - git subtree add --prefix=.claude/instructions/shared REGISTRY_URL v1.0.0
  - или symlink: ln -s ~/harness/instructions/shared .claude/instructions/shared

ФОРМАТ каждой инструкции:
  ---
  name: [name]
  type: local | shared
  trigger: glob:**/*.tsx | phase:w2 | manual
  tags: [react, ui, testing]
  ---
  # [Title]
  ## Context
  ## Rules
  ## Examples

VALIDACIYA:
  test -d .claude/instructions/local
  test -d .claude/instructions/shared
  [ "$(find .claude/instructions/local -name "*.md" | wc -l)" -ge 2 ]
  [ "$(find .claude/instructions/shared -name "*.md" | wc -l)" -ge 1 ]

---

## Модуль: Quality Pipeline
НЕОБХОДИМО: lefthook.yml + .github/workflows/ci.yml + linter config

ВЫБОР ИНСТРУМЕНТОВ (из стека):
  TypeScript -> Biome + ESLint + tsc + vitest
  Python -> ruff + mypy + pytest
  Go -> golangci-lint + go test

LOCAL (lefthook.yml):
  Копируй template/lefthook.yml, замени {{VARIABLES}}:
    {{LINT_CMD}} -> biome check (или eslint, или ruff check)
    {{TYPECHECK_CMD}} -> tsc --noEmit (или mypy)
    {{TEST_CMD}} -> vitest run (или pytest, или go test)
```

```
 CI (.github/workflows/ci.yml):
  Копируй template/ci.yml, замени {{VARIABLES}} аналогично.
  Добавь security-группу если requested:
    - semgrep scan
    - secret detection (truffleHog или git-secrets)

VALIDACIYA:
  test -f lefthook.yml
  test -f .github/workflows/ci.yml
  test -f biome.json || test -f eslint.config.js || test -f pyproject.toml
  [ "$(grep -c "run:" lefthook.yml)" -ge 3 ]
  [ "$(grep -c "runs-on:" .github/workflows/ci.yml)" -ge 3 ]

---

## Модуль: Knowledge Base
НЕОБХОДИМО: docs/knowledge/ с шаблонами

ДЕЙСТВИЕ: скопируй все файлы из template/knowledge/:
  cp template/knowledge/.template-*.md docs/knowledge/

СТРУКТУРА директории:
  docs/knowledge/
    .template-decision.md
    .template-lesson.md
    .template-pattern.md
    [NNN-slug.md — создаешь по мере работы]

VALIDACIYA:
  test -d docs/knowledge
  test -f docs/knowledge/.template-decision.md
  test -f docs/knowledge/.template-lesson.md

---

## Модуль: Session Protocol
ТИП: Embedded (раздел в AGENTS.md + feature_list.json)

ДЕЙСТВИЕ:
  1. Встраивай раздел ## Session Protocol в AGENTS.md (см. выше)
  2. Создавай feature_list.json:
     {
       "project": "[имя из package.json]",
       "features": [],
       "tech_debt": [],
       "active_decisions": [],
       "last_updated": "[today]"
     }

VALIDACIYA:
  grep -q "^## Session Protocol" AGENTS.md
  test -f feature_list.json
  python3 -c "import json; json.load(open('feature_list.json'))"

---

## Модуль: Skills Registry
```

```
 ЕСЛИ пользователь дал URL репозитория:
  git subtree add --prefix=.claude/instructions/shared URL v1.0.0
ИНАЧЕ:
  Создавай только local/, shared — позже.

---

## Модуль: Feedback Loop
ТИП: Process (раздел в AGENTS.md)

ДЕЙСТВИЕ: встраивай раздел ## Feedback Loop в AGENTS.md (см. выше).

VALIDACIYA:
  grep -q "^## Feedback Loop" AGENTS.md

---

## Post-Setup (после генерации ВСЕХ модулей)

1. Запусти: ./.harness/harness-validate.sh
2. Если ошибки — исправь и перезапусти.
3. После успеха — выведи summary:
   "Harness configured. Modules: 8. Errors: 0. Warnings: X.
    Manifest: .harness/manifest.json"
4. Закоммить: git add -A && git commit -m "harness: setup v1.0"
```

11. Матрица: Gen vs Template vs Validate

Таблица 3: Матрица генерации для каждого модуля

Модуль	Агент генерирует	Template копируется	Валидируется	Риск
AGENTS.md	Контент (stack, commands, anti-patterns)	Структура разделов	Размер <60, 6+ разделов	Low
Instruction Modules	local/*.md (по доменам проекта)	shared/ (git subtree)	2+ local, 1+ shared	Low
Quality Pipeline	Выбор инструментов под стек	lefthook.yml, ci.yml (структура)	3+ commands, 3+ jobs	Medium
Safety Config	— (embedded)	Фиксированный список запретов	5+ правил safety	None
Knowledge Base	— (создается по ходу)	3 шаблона (.template-*.md)	Директория + шаблоны	None
Session Protocol	feature_list.json	Структура раздела в AGENTS.md	Раздел + валидный JSON	Low
Skills Registry	— (infrastructure)	Git repo или symlink	Директория shared/	None
Feedback Loop	— (process)	Раздел в AGENTS.md	Раздел присутствует	None

Итого

3 модуля агент генерирует полностью (AGENTS.md, Instruction Modules local, feature_list.json). **3 модуля** копируются из templates (Knowledge Base templates, Quality Pipeline structure, Skills Registry). **2 модуля** embedded в AGENTS.md (Safety, Session Protocol, Feedback Loop). **Все 8** валидируются deterministic checker'ом.

Таблица 4: Риски генерации по модулю

Уровень риска	Модули	Митигация
None	Safety, Knowledge Base, Skills Registry, Feedback Loop	Не генерируются агентом

Low	AGENTS.md, Instruction Modules, Session Protocol	Validator ловит структурные ошибки
Medium	Quality Pipeline	Template структура + validator + CI gate

Ни один модуль не имеет высокого риска. Security-критичные элементы (CSP, secrets, auth) не генерируются агентом — они либо фиксированы в Safety-разделе, либо добавляются через опциональные capability-группы из verified templates.